Undecidability

THEORY OF COMPUTATION

In computability theory and computational complexity theory,

an undecidable problem is a decision problem for which it is proved to be impossible to construct an algorithm that always leads to a correct yes-or-no answer.

Decidability vs Undecidability

There are two types of TMs (based on halting):

Recursive: **TMs that always halt**, no matter accepting or non-accepting ≡ **DECIDABLE** PROBLEMS

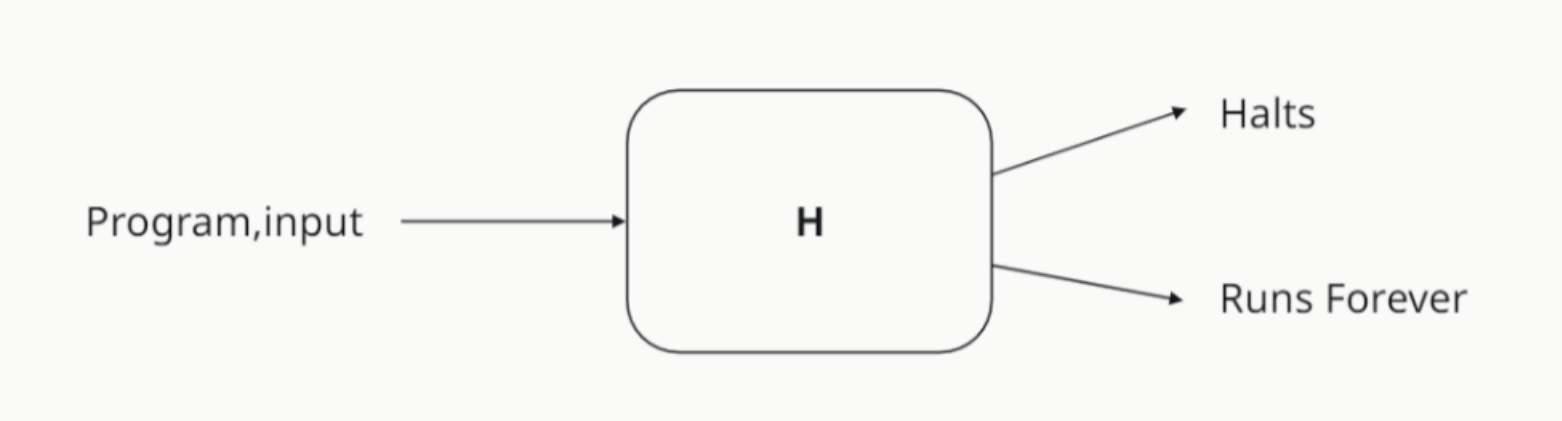
Recursively enumerable: **TMs that are guaranteed to halt only on acceptance**. If non-accepting, it may or may not halt (i.e., could loop forever).

Undecidability:

- Undecidable problems are those that are not recursive

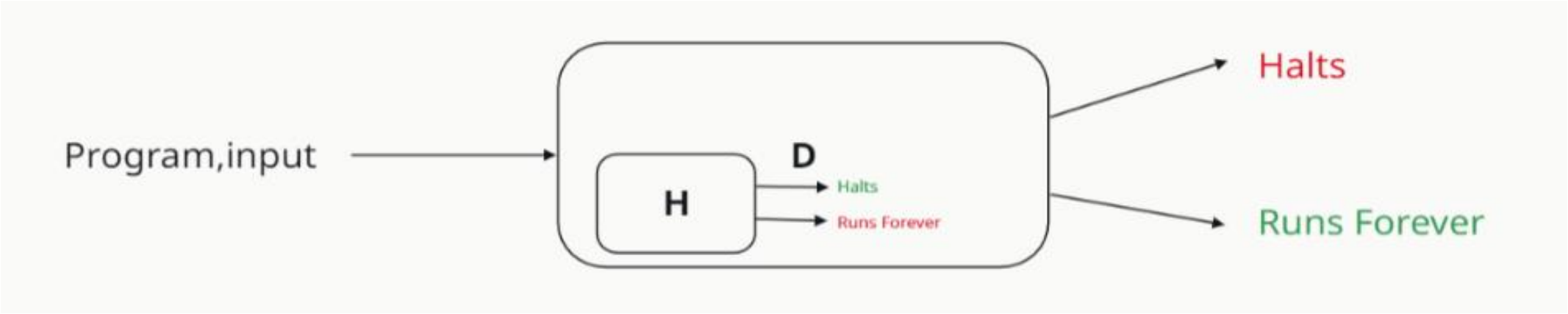
Halting Problem

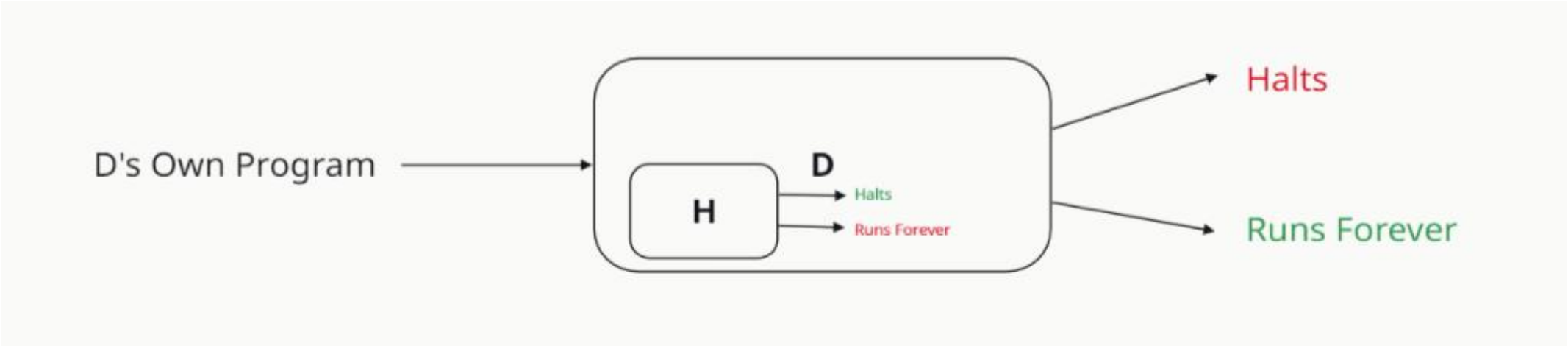
The Halting Problem asks whether there is a general algorithm that can examine any given computer program and input, and correctly determine whether the program will **eventually stop** (halt) or **loop forever**.



Let's H is always right
Proof by Contradiction!

Halting Problem Contd.





H can’t exist!!

SCOOPING THE LOOP SNOOPER
A proof that the Halting Problem is undecidable

Geoffrey K. Pullum
(*School of Philosophy, Psychology and Language Sciences, University of Edinburgh*)

No general procedure for bug checks will do.
Now, I won't just assert that, I'll prove it to you.
I will prove that although you might work till you drop,
you cannot tell if computation will stop.

For imagine we have a procedure called *P*
that for specified input permits you to see
whether specified source code, with all of its faults,
defines a routine that eventually halts.

You feed in your program, with suitable data,
and *P* gets to work, and a little while later
(in finite compute time) correctly infers
whether infinite looping behavior occurs.

If there will be no looping, then *P* prints out 'Good.'
That means work on this input will halt, as it should.
But if it detects an unstoppable loop,
then *P* reports 'Bad!' — which means you're in the soup.

Well, the truth is that *P* cannot possibly be,
because if you wrote it and gave it to me,
I could use it to set up a logical bind
that would shatter your reason and scramble your mind.

Here's the trick that I'll use — and it's simple to do.
I'll define a procedure, which I will call *Q*,
that will use *P*'s predictions of halting success
to stir up a terrible logical mess.

SCOOPING THE LOOP SNOOPER
A proof that the Halting Problem is undecidable

Geoffrey K. Pullum
(*School of Philosophy, Psychology and Language Sciences, University of Edinburgh*)

For a specified program, say *A*, one supplies,
the first step of this program called *Q* I devise
is to find out from *P* what's the right thing to say
of the looping behavior of *A* run on *A*.

If *P*'s answer is 'Bad!', *Q* will suddenly stop.
But otherwise, *Q* will go back to the top,
and start off again, looping endlessly back,
till the universe dies and turns frozen and black.

And this program called *Q* wouldn't stay on the shelf;
I would ask it to forecast its run on *itself*.
When it reads its own source code, just what will it do?
What's the looping behavior of *Q* run on *Q*?

If *P* warns of infinite loops, *Q* will quit;
yet *P* is supposed to speak truly of it!
And if *Q*'s going to quit, then *P* should say 'Good.'
Which makes *Q* start to loop! (*P* denied that it would.)

No matter how *P* might perform, *Q* will scoop it:
Q uses *P*'s output to make *P* look stupid.
Whatever *P* says, it cannot predict *Q*:
P is right when it's wrong, and is false when it's true!

I've created a paradox, neat as can be —
and simply by using your putative *P*.
When you posited *P* you stepped into a snare;
Your assumption has led you right into my lair.

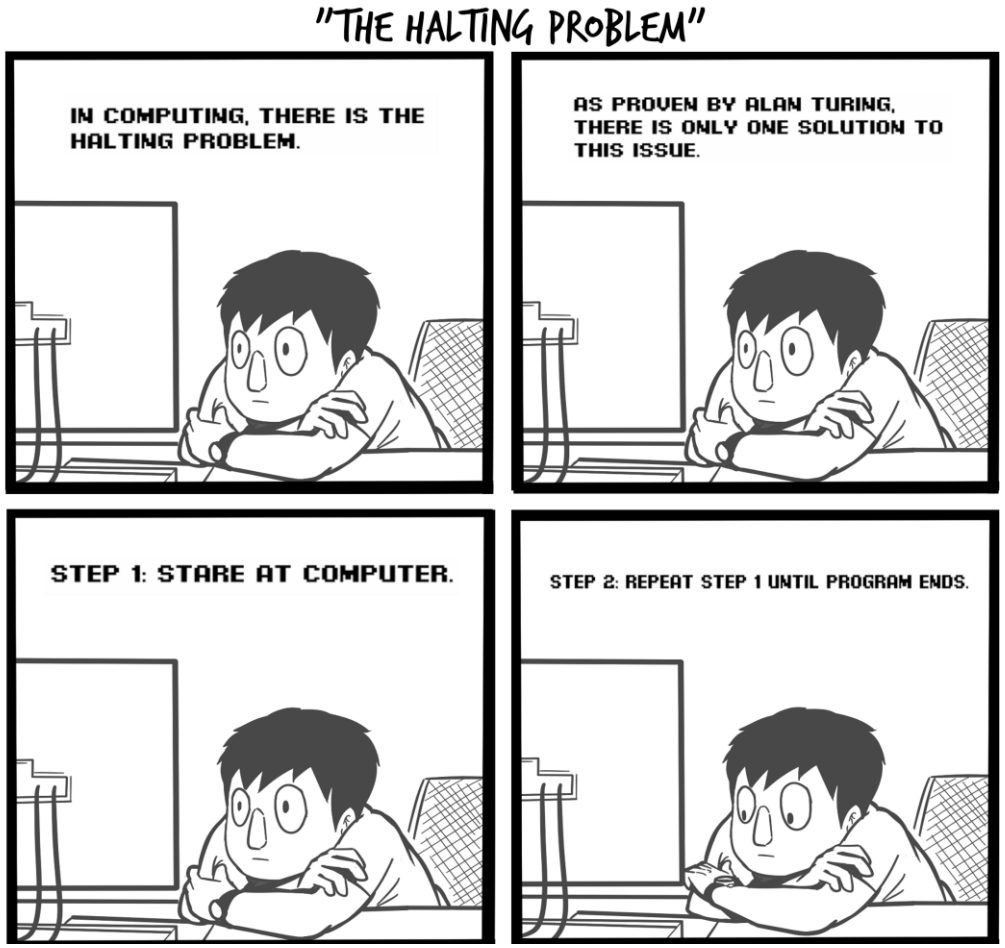
SCOOPING THE LOOP SNOOPER
A proof that the Halting Problem is undecidable

Geoffrey K. Pullum
(*School of Philosophy, Psychology and Language Sciences, University of Edinburgh*)

So where can this argument possibly go?
I don't have to tell you; I'm sure you must know.
We've proved that there just cannot possibly be
a procedure that acts like the mythical *P*.

You can never find general mechanical means
for predicting the acts of computing machines;
it's something that cannot be done. So we users
must find our own bugs. Our computers are losers!

Halting Problem Contd.



Another representation

```
def holds(fun):
    # can't be implemented in reality
    pass

def weird(fun):

    if holds(fun):
        while True:
            pass

    else:
        # Otherwise, halt immediately.
        return
```

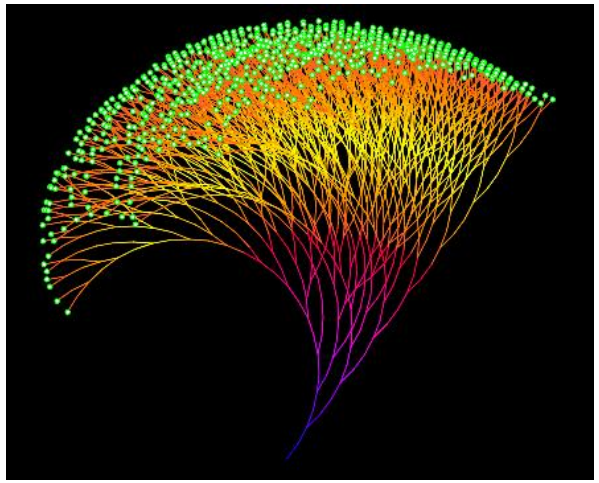
```
weirdstr = """
def weird(fun):
  if holds(fun):
    while True:
      pass
  else:
    return
"""

weird(weirdstr)
```

Collatz Conjecture!

1. Starting from any positive integer.
2. if it is even, divide it by 2
3. if it is odd, multiply it by 3 and add 1
4. The conjecture states that no matter what number you start with, you will eventually reach 1!

$$F(n) = \begin{cases} \frac{n}{2} & if\ n\%2 = 0 \\ 3n+1 & if\ n\%2 = 1 \end{cases}$$



References

http://www.lel.ed.ac.uk/%7Egpullum/loopsnoop.html

https://en.wikipedia.org/wiki/Collatz_conjecture